zeroriscy_compressed_decoder

OVERVIEW

The Compressed Instruction Decoder is responsible for converting 16-bit compressed RISC-V instructions into their equivalent standard 32-bit instruction format.
This module ensures compatibility between compressed instructions and the normal execution pipeline of the processor.
Since the processor execution stages operate using standard 32-bit instructions, this decoder expands compressed instructions before they proceed further in the pipeline.
The module is purely combinational, meaning it performs instruction decoding and expansion without requiring a clock signal. It mainly uses bit extraction, rearrangement, sign extension, and opcode insertion to reconstruct the correct 32-bit instruction format.

FUNCTIONAL DESCRIPTION: CORE RESPONSIBILITY

The primary function of this module is to detect whether the incoming instruction is compressed and, if required, convert it into a valid 32-bit RISC-V instruction.
This process includes:

- Identifying compressed instructions
- Expanding 16-bit instructions into 32-bit format
- Performing opcode insertion
- Rearranging immediate fields
- Applying sign extension where required
- Detecting illegal or unsupported instructions

This allows the processor to execute both compressed and standard instructions seamlessly.

INPUT AND OUTPUT SIGNALS

Input:

- instr_i [31:0] -> Input instruction fetched from instruction memory.

This input may contain either a normal 32-bit instruction or a 16-bit compressed instruction stored within the lower bits. The decoder first checks the instruction format before processing it.

Outputs:

- instr_o [31:0] -> Expanded 32-bit instruction output.

If the instruction is compressed, the decoder reconstructs and outputs the equivalent full 32-bit instruction. If already a normal instruction, it passes through without modification.

- is_compressed_o -> Compressed instruction indicator.

Active when the input is identified as compressed. Helps the processor track compressed instruction execution.

- illegal_instr_o -> Illegal instruction flag.

Asserted when the decoder detects invalid compressed formats, unsupported patterns, or reserved/illegal encodings. Ensures safe execution and proper exception handling.

WORKING PRINCIPLE: STEP 1 - INSTRUCTION TYPE DETECTION

The decoder first checks the two least significant bits of the instruction (instr_i[1:0]). These bits determine whether the instruction is compressed or standard.

Decision Logic:

- If the bits indicate a 16-bit compressed format -> Decode and expand
- Otherwise -> Treat as a normal 32-bit instruction

This forms the first stage of instruction classification.

WORKING PRINCIPLE: STEP 2 - COMPRESSED INSTRUCTION IDENTIFICATION

If the instruction is compressed, the module uses nested case statements to identify the exact compressed instruction type. Different compressed instruction groups are decoded based on opcode patterns and field positions.

Examples include:

- Compressed arithmetic instructions
- Compressed load/store instructions
- Compressed branch instructions
- Compressed jump instructions
- Compressed immediate operations

This structured decoding ensures accurate instruction reconstruction.

WORKING PRINCIPLE: STEP 3 - INSTRUCTION RECONSTRUCTION

Once the compressed instruction type is identified, the decoder rebuilds the equivalent 32-bit instruction using:

- Bit concatenation
- Field rearrangement
- Register mapping
- Immediate generation
- Opcode insertion

This process converts the compact 16-bit format into the standard RISC-V instruction format required by the execution stage.

WORKING PRINCIPLE: STEP 4 - SIGN EXTENSION

For instructions containing immediate values, sign extension is performed whenever necessary.

This ensures:

- Correct signed arithmetic
- Proper branch target calculation
- Accurate address generation

Without sign extension, execution results could become incorrect for negative values.

WORKING PRINCIPLE: STEP 5 - ILLEGAL INSTRUCTION DETECTION

If the instruction pattern does not match any valid compressed instruction format, the module sets:

illegal_instr_o = 1

This protects the processor from executing invalid instructions and supports exception handling mechanisms.

INTERNAL DESIGN APPROACH

Purely Combinational Logic:

The entire decoder is implemented using combinational logic. This means:

- No clock dependency
- Immediate instruction expansion
- Faster instruction delivery to the pipeline

This improves processor efficiency and reduces latency during instruction fetch and decode stages.

Bit Rearrangement Technique:

Compressed instructions store fields in compact formats. The decoder extracts these fields and rearranges them into standard RISC-V instruction positions. This includes:

- Immediate fields
- Register addresses
- Function fields
- Opcode fields

This technique enables correct translation without additional storage elements.

Nested Case-Based Decoding:

The use of nested case statements allows:

- Clear instruction classification
- Efficient decoding logic
- Easy support for multiple compressed instruction formats

This improves readability and maintainability of the RTL design.

KEY DESIGN ADVANTAGES

- Supports compressed RISC-V instruction execution
- Reduces instruction memory usage
- Improves code density
- Fully combinational for fast decoding
- Detects illegal instructions reliably
- Maintains compatibility with standard 32-bit execution pipeline

SUMMARY

The zeroriscy_compressed_decoder is an essential front-end module that enables support for compressed RISC-V instructions by converting them into standard 32-bit instructions.

It:

- Detects compressed instructions
- Expands them into full 32-bit format
- Performs sign extension and opcode insertion
- Flags illegal instructions when required
- Operates entirely using combinational logic for fast execution

This module improves instruction memory efficiency while ensuring smooth integration with the Zero-Riscy processor pipeline.